

Question No:7

Problem Description:

Any sequence A of size n is called **B-sequence** if:

$A_1 < A_2 < \dots < A_k > A_{k+1} > A_{k+2} > \dots > A_n$ where $1 \leq k \leq n$. That is, a sequence which is initially strictly increasing and then strictly decreasing (the decreasing part may or may not be there).

All elements in A except the maximum element comes atmost twice (once in increasing part and once in decreasing part) and maximum element comes exactly once.

All elements coming in decreasing part of sequence should have come once in the increasing part of sequence.

You are given a B-sequence S and Q operations. For each operation, you are given a value val . You have to insert val in S if and only if after insertion, S still remains a B-sequence .

After each operation, print the size of S . After all the operations, print the sequence S .

Hint: Think of using some data structure to support insertion of elements in complexity better than linear.

Input Constraints:

$1 \leq N \leq 10^5$
 $1 \leq S_i \leq 10^9$
 $1 \leq Q \leq 10^5$
 $1 \leq val \leq 10^9$
Given sequence S is a **B-sequence**,

Input Format:

First line consists of an integer N , denoting size of S .
Second line consists of N space separated integers, denoting elements of S .

Next line consists of an integer Q , denoting number of operations.
Each of the following Q lines consists of an integer val .

Output Format:

After each operation, print the size of S in a new line.
After all operations, print the sequence S .

✓ Logical Test Cases

Test Case 1

INPUT (STDIN)

4
1 2 5 2
4
5
1
3
2

EXPECTED OUTPUT

4
5
6
6
1 2 3 5 2 1

Test Case 2

INPUT (STDIN)

5
3 4 5 1 3
3
6
2
3
1

EXPECTED OUTPUT

6
7
7
1 2 3 4 5 6 3

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  typedef long long ll;
5
6  /* Compare function for qsort */
7  int compare(const void *a, const void *b)
8  {
9      ll x = *(const ll *)a;
10     ll y = *(const ll *)b;
11
12     if (x < y)
13         return -1;
14     if (x > y)
15         return 1;
16     return 0;
17 }
18
19 /* Binary search */
20 int binarySearch(ll arr[], int n, ll x)
21 {
22     int left = 0;
23     int right = n - 1;
24
25     while (left <= right)
26     {
27         int mid = left + (right - left) / 2;
```

```
29         if (arr[mid] == x)
30             return mid;
31
32         if (arr[mid] < x)
33             left = mid + 1;
34         else
35             right = mid - 1;
36     }
37
38     return -1;
39 }
40
41 int main()
42 {
43     int N;
44     scanf("%d", &N);
45
46     ll *S = (ll *)malloc(N * sizeof(ll));
47
48     for (int i = 0; i < N; i++)
49     {
50         scanf("%lld", &S[i]);
51     }
```

```
53     int Q;
54     scanf("%d", &Q);
55
56     ll *queries = (ll *)malloc(Q * sizeof(ll));
57
58     for (int i = 0; i < Q; i++)
59     {
60         scanf("%lld", &queries[i]);
61     }
62
63     /*
64      * Coordinate compression.
65      *
66      * We need all values that can ever appear:
67      * - Initial sequence values
68      * - Query values
69      */
70
71     int total = N + Q;
72
73     ll *all = (ll *)malloc(total * sizeof(ll));
74
75     for (int i = 0; i < N; i++)
76         all[i] = S[i];
77
78     for (int i = 0; i < Q; i++)
79         all[N + i] = queries[i];
```

```

81     qsort(all, total, sizeof(ll), compare);
82
83     /* Remove duplicates */
84     int M = 0;
85
86     for (int i = 0; i < total; i++)
87     {
88         if (i == 0 || all[i] != all[i - 1])
89         {
90             all[M++] = all[i];
91         }
92     }
93
94     /*
95      |   |   |   |
96      |   |   |   |   freq[i] = how many times the value all[i]
97      |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
98      */
99     int *freq = (int *)calloc(M, sizeof(int));
100
101     /* Current maximum value */
102     ll maxValue = S[0];
103
104     /* Count initial elements */
105     for (int i = 0; i < N; i++)
106     {
107         int pos = binarySearch(all, M, S[i]);
108         freq[pos]++;

```

```

108
109     if (S[i] > maxValue)
110         maxValue = S[i];
111 }
112
113 int currentSize = N;
114
115 /*
116  Process queries
117 */
118 for (int i = 0; i < Q; i++)
119 {
120     ll val = queries[i];
121
122     int pos = binarySearch(all, M, val);
123
124
125     if (freq[pos] >= 2)
126     {
127         /* Invalid */
128     }
129     else if (val == maxValue && freq[pos] == 1)
130     {
131         /* Invalid */
132     }
133     else
134     {

```

```

135         /* Valid insertion */
136         freq[pos]++;
137         currentSize++;
138
139         if (val > maxValue)
140             maxValue = val;
141     }
142
143     /* Print size after every operation */
144     printf("%d\n", currentSize);
145 }
146
147 /*
148     Print final B-sequence.
149
150     First:
151     Increasing part
152
153     Then:
154     Decreasing part containing values
155     that occur twice.
156 */
157
158 for (int i = 0; i < M; i++)
159 {
160     if (freq[i] >= 1)

```

```

161     {
162         printf("%lld", all[i]);
163
164         printf(" ");
165     }
166 }
167
168
169 for (int i = M - 1; i >= 0; i--)
170 {
171     if (freq[i] == 2)
172     {
173         printf("%lld", all[i]);
174
175         if (i != 0)
176             printf(" ");
177     }
178 }
179
180 printf("\n");
181
182 free(S);
183 free(queries);
184 free(all);
185 free(freq);
186
187     return 0;
188 }

```

Output:

Test Case-1:

```

4
1 2 5 2
4
5
1
3
2
4
5
6
6
1 2 3 5 2 1

```

Test Case-2:

	5	
	3 4 5 1 3	
	3	
	2	
	3	
	1	
	6	
	6	
	7	
	1 2 3 4 5 3 1	

Keshav Kumar Rauniyar
CH.SC.U4CSE25236